

Python 内置函数手册（教科书配套版）

打印提示：A4、实际大小（100%）、**关闭浏览器"页眉和页脚"**——本套教材的 PDF 版已自带页码，重复勾选会产生双页码；需要保留彩色底色时，在打印选项里开启"背景图形"。

目录

理念篇	先搞懂函数是什么，再记函数（第一页必读）
第 1 组	输入输出：print / input
第 2 组	类型工厂：str/int/float/bool/list/tuple/dict/set/type + 切片速查
第 3 组	长度与统计：len/sum/min/max/abs/round
第 4 组	序列工具：range/enumerate/zip/sorted/reversed
第 5 组	检查工具：isinstance/dir/help
第 6 组	工具集：pow/divmod/chr/ord/format/all/any/repr + os/time 常用函数 + 自定义函数速查
文件与 JSON	JSON 四件套、文件对象三连（运维天天用）
文件对象三连	open / read / with（三种文件操作速查）
字符串方法速查	11 条必会：split/join/strip(带参)/replace/大小写/find/头尾判断/count/rsplit/splitlines/isdigit
组合练习	5 道函数组合套路题（含答案）
替代写法对比	f-string vs %；subprocess vs os.system；logging vs print
章末作业	8 题（答案紧随题目，先做再看）
练习答案	先做再看（按组汇总）
识别题答案	按组速览
尾页·记忆索引	分组速查用

理念篇：先搞懂函数是什么，再记函数

1. 函数 = 一台加工机器

进去的是**原料**（参数），出来的是**产品**（返回值）：

```
len("hello") # 原料 "hello" → 机器 len → 产品 5
```

2. 三条黄金规律

★ 规律①：**名字即用途**。print打印、input输入、len长度(length)、sum求和、sorted排序——英文单词看懂，函数就懂一半。

★ 规律②：**括号是开关**。函数名后面跟 () 才开工；不写括号只是把机器指给你看。

★ 规律③：**函数 vs 方法**。函数是 len(x)（手拿工具加工）；方法是 x.split()（工具长在对象身上，点号调用）。

3. 最重要的两个函数（第一天就认识）

函数	作用	示例
print()	打印到屏幕	print("a", "b", sep="-", end="!") → a-b!
input()	读键盘输入	name = input("你叫什么: ") → 停下等你输入；回车后 name = 你输入的文字

⚠ input() 拿到的永远是字符串！要算数先 int()。

4. 类型转换家族 = "工厂函数"

str() int() float() bool() list() tuple() dict() set() —— 都是把一种类型"加工"成另一种类型的工厂。见第 2 组。

5. 变量 = 贴了标签的盒子

```
age = 19 # 造一个盒子，贴上"age"标签，把 19 放进去。以后写 age 就等于是 19。
```

命名规律：见名知意（age、name、disk_usage）；英文小写，多词用下划线连接。

第 1 组 · 输入输出 (print / input)

函数	作用	示例	注意点
<code>print()</code>	打印；多参数默认空格分隔	<code>print("a", "b")</code> → a b <code>print("a", "b", sep="-")</code> → a-b <code>print("hi", end="")</code> → hi (不换行)	sep=分隔符 end= 结尾 (默认换行)
<code>input()</code>	读键盘，返回字符串	<code>name = input("你叫什么: ")</code> → 停下等你输入；回车后 <code>name = 你输入的文字</code>	返回永远是 str !

🔗 识别题 (先猜, 再看答案)

- `print("a", "b", sep="")` 输出?
- 用户输入 42, `name = input("?")` 后, `name` 的类型?

第 2 组 · 类型工厂 (含 type 与切片速查)

函数	作用	示例	注意点
<code>str()</code>	转字符串	<code>str(42)</code> → "42"	拼接字符串前常用
<code>int()</code>	转整数	<code>int("42")</code> → 42 <code>int("3.14")</code> → 报错	转不了会报 ValueError
<code>float()</code>	转小数	<code>float("3.14")</code> → 3.14	
<code>bool()</code>	转真假	<code>bool(0)</code> → False <code>bool("x")</code> → True	空的东西多为 False
<code>list()</code>	转列表	<code>list("abc")</code> → ['a', 'b', 'c']	
<code>tuple()</code>	转元组 (不可变)	<code>tuple([1,2])</code> → (1, 2)	
<code>dict()</code>	造字典	<code>dict(a=1)</code> → {'a': 1}	
<code>set()</code>	造集合 (自动去重)	<code>set([1,1,2])</code> → {1, 2}	无序
<code>type()</code>	查类型	<code>type(5)</code> → <class 'int'> <code>type("5")</code> → <class 'str'>	排查"类型不对"第一招; 显示为 <class 'xxx'> 形式

🔗 **切片速查** (字符串/列表通用) : `x[起点:终点:步长]`, 终点**不含**。"python"[0:2]→"py"、[-1]最后一个、[::-1]反转。

🔗 识别题

- `int("42") + 8 = ?`
- `list("abc") = ?`
- `"python"[1:4] = ?`

第 3 组 · 长度与统计 (len/sum/min/max/abs/round)

函数	作用	示例	注意点
len()	长度	len("hello") → 5 len([1,2]) → 2	字符串/列表/字典通用
sum()	求和	sum([1,2,3]) → 6	
min() max()	/ 最小/最大	min([3,1,2]) → 1 max("abc") → 'c'	
abs()	绝对值	abs(-5) → 5	
round()	保留小数位	round(3.14159, 2) → 3.14	⚠ 银行家舍入： round(2.5)→2、round(3.5)→4 (四舍六入五成双)

🔗 识别题

1. round(2.5) = ?
2. len([1, 2, 3]) = ?

第 4 组 · 序列工具 (range/enumerate/zip/sorted/reversed)

函数	作用	示例	注意点
range()	生成数列	list(range(5)) → [0, 1, 2, 3, 4] list(range(1,10,2)) → [1, 3, 5, 7, 9] range(5) → range(0, 5)	终点不含；直接敲 range(5) 显示的是对象本身 range(0, 5)，要看数字得套 list()
enumerate()	编号遍历	for i, x in enumerate(xs, 1): → i 从 1 开始编号（不写第二个参数则从 0）	第二个参数=起始编号
zip()	拉链配对	list(zip([1,2], ["a","b"])) → [(1, 'a'), (2, 'b')]	zip() 本身返回迭代器，要列表得套 list()
sorted()	排序（返回新列表）	sorted([3,1,2]) → [1, 2, 3] reverse=True 倒序	原列表不变
list.sort()	原地排序（改原列表，返回 None）	a = [3,1]; a.sort(); print(a) → [1,3] 与 sorted() 的区别：sort() 改原列表不返回新列表；sorted() 返回新列表不动原列表（进阶篇已统一用 sorted()）	
reversed()	反向	list(reversed([1,2])) → [2, 1]	要 list() 包一层

🔗 识别题

1. list(range(1, 10, 2)) = ?
2. sorted([3,1,2], reverse=True) = ?

第 5 组 • 检查工具 (isinstance/dir/help)

函数	作用	示例	注意点
<code>isinstance()</code>	判断是不是某类型	<code>isinstance(5, int) → True</code> <code>isinstance("5", int) → False</code>	比 <code>type</code> 更推荐 (有继承概念时更稳)
<code>dir()</code>	列出对象所有方法	<code>dir("abc") → ['__add__', 'split', 'strip', 'upper', ...]</code>	真实输出很长 (含双下划线开头的方法), 此处为示意省略, 以你机器实际输出为准; 想不起字符串能干什时
<code>help()</code>	看说明书	<code>help(len) → 交互式英文文档 (内容以实际输出为准, 按 q 退出)</code>	等于 Linux 的 <code>man</code>

🔗 识别题

- `isinstance("5", int) = ?`
- `dir("abc")` 返回什么?

第 6 组 • 工具集 (数学杂项 + all/any/repr + os/time 常用 + 自定义函数速查)

6.1 数学与杂项

函数	作用	示例	注意点
<code>pow()</code>	幂	<code>pow(2, 3) → 8</code>	<code>= 2**3</code>
<code>divmod()</code>	商和余数	<code>divmod(10, 3) → (3, 1)</code>	一次拿两个
<code>chr()</code> <code>ord()</code>	/ 字符↔编码	<code>ord("A") → 65</code> <code>chr(65) → 'A'</code>	
<code>format()</code>	格式化数字	<code>format(3.14159, ".2f") → '3.14'</code>	f-string 更方便
<code>all()</code> <code>any()</code>	/ 全真 / 有真	<code>all([1,2,0]) → False</code> <code>any([0,0,1]) → True</code>	0、""、None 算"假"
<code>repr()</code>	看值的"原始表示"	<code>repr("hi") → "'hi'" (带引号)</code>	调试时区分类型用

⚠ `eval()` / `exec()` 是危险函数: 能把字符串当代码执行。网上很多新手教程用它, 但生产环境禁用——永远别用它们处理用户输入。

6.2 os / time 常用函数速查 (运维最常用)

函数	作用	示例
os.getcwd()	当前目录	<code>import os; os.getcwd()</code> → '/home/你的用户名'
os.listdir(路径)	列目录内容	<code>os.listdir(".")</code> → 当前目录的文件名列表 (内容以你机器实际为准)
os.path.join(a, b)	拼路径 (自动用对分隔符)	<code>os.path.join("home", "student")</code> → "home/student"
os.rename(旧, 新)	重命名/移动文件	<code>os.rename("a.txt", "b.txt")</code> → 不报错就是成功 (旧名消失、新名出现)
os.path.exists(路径)	判断文件是否存在	<code>os.path.exists("a.txt")</code> → True/False
os.path.getsize(路径)	文件大小 (字节)	<code>os.path.getsize("a.txt")</code> → 数字 (大小以你机器的文件为准)
os.path.getmtime(路径)	最后修改时间 (时间戳)	<code>os.path.getmtime("a.txt")</code> → 数字 (时间戳, 值以你机器为准)
time.localtime(秒)	时间戳 → 可读时间	<code>time.strftime("%F", time.localtime(时间戳))</code> → '2026-08-25' (示例)
time.time()	时间戳 (1970 起的秒数)	<code>import time; time.time()</code> → 1758400000.0 (示例值, 每次运行都在涨)
time.sleep(秒)	暂停	<code>time.sleep(1)</code> → 暂停 1 秒后继续 # 脚本里"等一会"用
time.strftime(格式)	格式化当前时间	<code>time.strftime("%Y-%m-%d %H:%M:%S")</code> → '2026-09-21 14:30:00' (示例)
random.randint(a, b)	随机整数 (a~b, 两头都包含)	<code>import random; random.randint(1, 6)</code> → 1~6 的随机数

补一个字节串方法: "你好".encode("utf-8") 得到字节串 (b'...' 形式, 这是合法写法——bytes 字面量只能直接写 ASCII, 中文要先 encode); subprocess 不设 text=True 时输出就是字节串, 用 .decode("utf-8") 转回字符串 (进阶篇场景 5 用到)。

6.3 自定义函数结构速查

下面是结构示意 (伪代码, 不可直接运行); 每个中文占位换成真实名字

```
def 函数名(参数):  
    # 注释: 这函数干什么  
    结果 = 参数 加工过程  
    return 结果          # 把结果交回来; 忘写 return → 返回 None
```

返回值 = 函数名(实参) # 调用: 函数名 + 括号才开工

🔗 识别题

- `divmod(10, 3) = ?`
- `all([1, 2, 0]) = ?`
- Linux 上 `os.path.join("a", "b") = ?`

文件与 JSON：速查

json 四件套（配置文件 / API 数据专用，教科书第 6 章）：

写法	作用
<code>json.load(f)</code>	从文件对象读 JSON → dict
<code>json.dump(d, f, ensure_ascii=False)</code>	把 dict 写进文件（ <code>ensure_ascii=False</code> 让中文不乱码） 先 <code>open</code> 文件再调用 → 写入成功、不返回内容
<code>json.loads(字符串)</code>	从字符串解析 → dict
<code>json.dumps(d)</code>	把 dict 变成 JSON 字符串 <code>json.dumps({"a": 1})</code> → <code>'{"a": 1}'</code>

文件对象三连（open / read / write / with）

 **准备材料（先照抄跑一次）：**下面所有示例都读写同一个 `a.txt`，先创建它并写入两行：

```
with open("a.txt", "w", encoding="utf-8") as f:
    f.write("第一行\n第二行\n")  # w 模式：不存在就创建（存在会清空，正好保证每次起点一致）
```

从空工作目录开始照抄：先跑这个准备框，再依次跑后面的 `r / readlines / w / a` 示例，任何一步都不会因文件不存在而中断。

```
with open("a.txt", "r", encoding="utf-8") as f:  # with = 自动关门，推荐写法
    内容 = f.read()  # 全读成一整个字符串
# △ 同一文件对象读完一次就到末尾：read 之后再 readlines 通常得到空列表；
# 要么二选一，要么重新 open，或 f.seek(0) 回到开头再读。
```

```
with open("a.txt", "r", encoding="utf-8") as f:
    行列表 = f.readlines()  # 按行读成列表
```

```
with open("a.txt", "w", encoding="utf-8") as f:
    f.write("内容\n")  # 写；\n = 换行
```

```
with open("a.txt", "a", encoding="utf-8") as f:
    f.write("追加一行\n")  # 追加：不清空原文
```

模式	含义	对照 Linux
"r"	读	cat
"w"	写（  打开即清空原文件）	>（覆盖）
"a"	追加（不清空）	>>

识别题

- 已有内容的文件用 "w" 打开会怎样？
- with open 比手动 open+close 好在哪？

字符串方法速查 (11 个必会方法)

方法	作用	示例
<code>s.split(符号)</code>	切开成列表	<code>"a,b,c".split(",")</code> → ['a', 'b', 'c']
<code>"符号".join(列表)</code>	列表拼成字符串	<code>"-".join(["a","b"])</code> → 'a-b'
<code>s.strip()</code> / <code>strip("字符")</code> / <code>lstrip()</code> / <code>rstrip()</code>	去首尾空白; 带参数=去首尾指定的字符	<code>" hi ".strip()</code> → 'hi' <code>"%80%".strip("%")</code> → '80' <code>" .jpg".lstrip(".")</code> → 'jpg'
<code>s.replace(a, b)</code>	替换	<code>"a-b".replace("-", "_")</code> → 'a_b'
<code>s.upper()</code> / <code>lower()</code>	大小写	<code>"ab".upper()</code> → 'AB'
<code>s.find(x)</code>	找位置, 找不到返回 -1	<code>"abc".find("b")</code> → 1
<code>s.startswith(x)</code> / <code>endswith(x)</code>	判断开头/结尾	<code>"a.log".endswith(".log")</code> → True
<code>s.count(x)</code>	数出现次数	<code>"abab".count("a")</code> → 2
<code>s.rsplit(符号, 次数)</code>	从右边切 (取扩展名/最后一段)	<code>"a.b.c".rsplit(".", 1)[-1]</code> → 'c'
<code>s.isdigit()</code>	判断是不是纯数字	<code>"123".isdigit()</code> → True <code>"12a".isdigit()</code> → False
<code>s.splitlines()</code>	按行切成列表 (读命令输出必用)	<code>"a\nb".splitlines()</code> → ['a', 'b']

识别题

- `"a,b,c".split(",")` = ?
- `" hello ".strip()` = ?

组合练习（5 道套路题，答案附后）

学函数不是背单个，而是会**组合**。这 5 道是运维脚本里出现频率最高的组合套路。

1. 统计一个日志文件里 "error" 出现几次 (open + read + count)
2. 求 [85, 92, 78, 90] 的平均分 (sum + len)
3. 把用户输入的一串数字（空格分隔）从小到大排序打印 (input + split + int + sorted, 用循环转换)
4. 从一行日志 "2026-08-25 14:30:12 INFO 服务启动" 里只取出日期部分 (split 或切片)
5. 算 1 加到 100 的和 (range + sum)

替代写法对比（三组，各有推荐）

写法 A（老）	写法 B（新，推荐）	为什么推荐 B
"我叫%s, %d岁" % (name, age)	f"我叫{name}, {age}岁"	f-string 直观、不易错位
os.system("ls")	subprocess.run(["ls"], capture_output=True, text=True)	能拿输出、能判返回值、无命令注入风险
print("调试信息")	logging.info("调试信息")	日志带时间/级别、能写文件、可开关

章末作业（8 题，先做再看答案）

1. 把 "apple,banana,cherry" 切成列表，打印第 2 个元素
2. 输入年龄，判断是否成年 (≥ 18 输出"成年")
3. 把 [1, 2, 3] 反转并求和
4. 用 enumerate 给自己的 3 条待办事项加编号打印（从 1 开始）
5. 用字典存 "hello:你好"，用 get 查 "goodbye"（给默认提示）

6. 用 os.path.join 拼出 "logs/2026/08" 这样的路径
7. 实测 round(2.5) 和 round(3.5)，记录结果并解释
8. 用 format 和 f-string 各输出一次 π 保留两位小数

作业答案：

```
# 1
fruits = "apple,banana,cherry".split(",")
print(fruits[1])           # banana

# 2
age = int(input("年龄: "))
print("成年" if age >= 18 else "未成年")

# 3
xs = [1, 2, 3]
print(sum(reversed(xs)))   # 6（反转后还是 1+2+3，但机制要会）

# 4
todos = ["学 Linux", "学 Python", "写日志"]
for i, t in enumerate(todos, 1):
    print(f"{i}. {t}")

# 5
d = {"hello": "你好"}
print(d.get("goodbye", "查无此词"))

# 6
import os
path = os.path.join("logs", "2026", "08")
print(path)                # logs/2026/08 (Linux)

# 7
print(round(2.5), round(3.5))  # 2 4 — 银行家舍入：5前面是偶数→舍，奇数→进

# 8
pi = 3.14159
print(format(pi, ".2f"))      # 3.14
print(f"{pi:.2f}")           # 3.14
```

练习答案汇总（先做再看）

识别题与组合练习的答案全部集中在这里。对完答案请回看题目，把做错的重新想一遍。

识别题答案（按组）

第1处：答案：1. ab（分隔符是空串） 2. str（字符串，永远不是数字——要算数先 int()）

第2处：答案：1. 50（先转整数再相加） 2. ['a','b','c'] 3. "yth"（1、2、3 位，不含 4）

第3处：答案：1. 2（不是 3！银行家舍入：5 前面是偶数 2 → 舍） 2. 3

第4处：答案：1. [1,3,5,7,9] 2. [3,2,1]

第5处：答案：1. False（"5" 是 str） 2. 字符串能用的所有方法名列表（split、strip.....）

第6处：答案：1. (3, 1) 2. False（0 算假） 3. "a/b"（Windows 上是 "a\b"——这就是用 join 别手拼的理由）

第7处：答案：1. 立刻清空（重要文件先备份或用 "a"） 2. 自动关闭文件，不会忘写 close

第8处：答案：1. ['a','b','c'] 2. "hello"（读文件后每行都该 strip 一下，去掉末尾换行）

组合练习参考答案

参考答案（运行前先准备：题 1 先 echo 'error' > log.txt 建文件；题 2 自行输入三个数字；题 3 交互式输入数字串；其余按题意准备）

```
# 1
with open("log.txt", encoding="utf-8") as f:
    print(f.read().count("error"))

# 2
scores = [85, 92, 78, 90]
print(sum(scores) / len(scores)) # 86.25

# 3
parts = input("输入数字（空格分隔）：").split()
nums = []
for p in parts:
    nums.append(int(p))
print(sorted(nums))

# 4
line = "2026-08-25 14:30:12 INFO 服务启动"
print(line.split()[0]) # 2026-08-25（split后取第0块）
print(line[:10]) # 2026-08-25（切片前10个字符）

# 5
print(sum(range(1, 101))) # 5050
```

- ① **看名字**：print打印、input输入、len长度、sum求和、sorted排序
- ② **变类型**：str/int/float/bool/list/tuple/dict/set——工厂函数全家桶
- ③ **量一量**：len长度、sum求和、min小、max大、abs绝对值、round注意银行家舍入
- ④ **排一排**：range数列、enumerate编号、zip配对、sorted排序、reversed反转
- ⑤ **查一查**：type/isinstance看类型、dir看方法、help看说明书
- ⑥ **文件三连**：open开门、read读、write写，with自动关门
- ⑦ **字符串方法（11个）**：split切、join拼、strip洗（可带参数）、replace换、upper/lower大小写、find找、startswith/endswith看头尾、count数、rsplit右切、splitlines按行切、isdigit验数字



本册待改进清单（学习中发现手册不好用的地方，攒 5 条再统一改版）：

（留白：学习中发现手册哪里不好用就记在这里，攒满 5 条再统一改版）